Functional Programming

Liting Zhang

University of West Florida

Overview

- FP basics: expressions, immutability, no side effects
- Lambda expressions and higher-order functions
- Recursion as a core control mechanism
- Lists and pattern matching
- map, filter, reduce (fold)
- Strict vs lazy evaluation (conceptual)
- FP in the wild: functional style in TypeScript and Python
- Pure FP in Haskell: types, purity, IO boundary
- Industry angle: data pipelines, front-end state, concurrency

Learning goals

After this lecture, you should be able to

- Explain referential transparency, purity, and immutability
- Identify side effects and refactor toward pure functions
- Use lambdas and higher-order functions to build pipelines
- Write and reason about recursive definitions and base cases
- Use pattern matching to decompose data safely
- Compare strict and lazy evaluation and predict basic behavior
- Apply functional style in TypeScript and Python
- Explain how Haskell isolates effects with IO types
- Describe why FP helps with data flow, state management, and concurrency

Core idea: program by transforming values

A useful FP mental model:

- Data flows through a sequence of transformations
- Each step is ideally a pure function: input values to output values
- Effects exist, but are pushed to the edges: reading input, writing output, network calls

Typical benefits:

- Easier reasoning: fewer hidden dependencies
- Easier testing: pure functions are simple to test
- Easier parallelism: immutable data reduces data races

FP basics

Expressions and values

FP emphasizes expressions.

- Expression: evaluates to a value
- Statement: performs an action (often for effects)

FP style tends to:

- Prefer expression-oriented design
- Use function return values rather than updating shared variables
- Build programs by composing expressions

Purity and side effects

Pure function

- Same inputs always produce the same output
- No observable side effects

Side effects include:

- Mutating objects or global variables
- Printing, logging
- Reading time, randomness
- I/O: files, network, database

Pure functions are not always possible, but they are often the best default for core logic.

Referential transparency

Referential transparency means an expression can be replaced by its value without changing program behavior.
If f is pure, then:

- Let y = f(10)
- You can replace every f(10) with y safely

Why it matters:

- Supports equational reasoning
- Makes refactoring and optimization safer
- Makes memoization correct

Immutability

Immutability means data does not change after creation.

- Updates produce new values rather than modifying old ones
- Old values remain valid and can be safely shared

Practical consequences:

- Fewer bugs from unexpected mutation
- Easier undo/redo and time-travel debugging in UI state
- Easier concurrency since shared reads are safe

TypeScript: mutation vs immutable update

Arrays and objects are mutable by default, but FP style prefers copying on update.

```ts
type User = { id: string; name: string; points: number };

const u: User = { id: "u1", name: "Ada", points: 10 };

// mutation
u.points = u.points + 5;

// immutable update (copy)
const u2: User = { ...u, points: u.points + 5 };

console.log(u.points);
console.log(u2.points);
```

In FP style, core logic prefers producing u2 rather than mutating u.

TypeScript: readonly helps communicate intent

readonly reduces accidental mutation in larger codebases.

```
type User = { readonly id: string; readonly name: string;
↪  readonly points: number };

function awardPoints(u: User, delta: number): User {
  return { ...u, points: u.points + delta };
}

const u1: User = { id: "u1", name: "Ada", points: 10 };
const u2 = awardPoints(u1, 5);

console.log(u1.points);
console.log(u2.points);
```

readonly is a compile-time constraint; it supports discipline in functional style.

Python: mutation vs immutable update

Python lists and dicts are mutable, tuples are immutable.

```python
u = {"id": "u1", "name": "Ada", "points": 10}

# mutation
u["points"] = u["points"] + 5

# immutable-style update (copy)
u2 = {**u, "points": u["points"] + 5}

print(u["points"])
print(u2["points"])
```

Copy-on-update is common for FP-style code in Python.

Effects at the edges

A common FP architecture idea:

- Keep business logic pure
- Put I/O and mutation in a small boundary layer

Typical layering:

- Pure domain functions: compute results from inputs
- Adapter layer: read files, call network, parse inputs
- Presentation layer: print or render outputs

This is not limited to Haskell; it is widely used in TypeScript and Python services.

Practice 1: identify side effects and refactor (TypeScript)

Identify which lines cause effects, then refactor so the core computation is pure.

```
1  type Cart = { items: number[]; total: number };
2
3  function addItem(cart: Cart, price: number): void {
4    cart.items.push(price);
5    cart.total = cart.total + price;
6    console.log("added", price);
7  }
8
9  const c: Cart = { items: [], total: 0 };
10 addItem(c, 10);
11 addItem(c, 5);
12 console.log(c.total);
```

- Which lines are side effects
- Refactor to a pure function that returns a new cart
- Keep logging separate from the pure update

Practice 1: identify side effects and refactor (TypeScript)

Side effects in the original code

- cart.items.push(price) mutates the items array
- cart.total = ... mutates cart state
- console.log is an I/O effect

```
type Cart = { items: number[]; total: number };
function addItemPure(cart: Cart, price: number): Cart {
  const items2 = [...cart.items, price];
  const total2 = cart.total + price;
  return { items: items2, total: total2 };
}
function logAdded(price: number): void {
  console.log("added", price);
}
let c: Cart = { items: [], total: 0 };
c = addItemPure(c, 10);
logAdded(10);
c = addItemPure(c, 5);
logAdded(5);
console.log(c.total);
```

Lambdas and higher-order functions

Key term: lambda

Lambda

- An anonymous function written inline without a name
- Used when a function is needed as a value
- Often used as an argument to higher-order functions

Same idea in three languages:

- TypeScript: $(x) \Rightarrow x * 2$
- Python: lambda x: x * 2
- Haskell: $\backslash x \rightarrow x * 2$

Higher-order functions

Higher-order function

- Takes a function as an argument, or
- Returns a function as a result

Why they are important in FP:

- Separate mechanism from policy
- Reuse control patterns (map, filter, fold)
- Enable composition of small functions into pipelines

TypeScript: functions as values

```
1  type Pred<T> = (x: T) => boolean;
2
3  function keep<T>(xs: T[], p: Pred<T>): T[] {
4    return xs.filter(p);
5  }
6
7  const nums = [1, 2, 3, 4, 5];
8  const evens = keep(nums, x => x % 2 === 0);
9
10 console.log(evens);
```

keep is higher-order because it takes p as an argument.

Python: functions as values

```
1 def keep(xs, p):
2     return [x for x in xs if p(x)]
3
4 nums = [1, 2, 3, 4, 5]
5 evens = keep(nums, lambda x: x % 2 == 0)
6
7 print(evens)
```

Haskell: higher-order functions are normal

Haskell uses functions everywhere, and types make the contracts explicit.

```haskell
keep :: (a -> Bool) -> [a] -> [a] -- signature
-- takes a predicate from a to Bool, then a list of a, and
 ↪  returns a list of a.
keep p xs = filter p xs -- definition

evens :: [Int]
evens = keep even [1,2,3,4,5]
```

even is a function from Int to Bool.

map, filter, reduce

Common list pipeline primitives:

- map: transform each element independently
- filter: keep elements that satisfy a predicate
- reduce/fold: combine elements into one result

These patterns occur in:

- data pipelines (ETL, analytics)
- UI list rendering and filtering
- stream processing and log processing

TypeScript: map, filter, reduce

```
1 const nums = [1, 2, 3, 4, 5];
2
3 const squares = nums.map(x => x * x);
4 const evens = nums.filter(x => x % 2 === 0);
5 const sum = nums.reduce((acc, x) => acc + x, 0);
6
7 console.log(squares);
8 console.log(evens);
9 console.log(sum);
```

The callback defines custom behavior; the iteration mechanism is reused.

Python: map/filter and comprehensions

Python supports map/filter, but comprehensions are often clearer.

```
nums = [1, 2, 3, 4, 5]

squares = [x * x for x in nums]
evens = [x for x in nums if x % 2 == 0]
total = sum(nums)

print(squares)
print(evens)
print(total)
```

Haskell: map, filter, fold

```haskell
nums :: [Int]
nums = [1,2,3,4,5]

squares :: [Int]
squares = map (\x -> x*x) nums

evens :: [Int]
evens = filter even nums

total :: Int
total = foldl (\acc x -> acc + x) 0 nums
```

foldl is a reduce; foldr is another common fold with different evaluation behavior.

Recursion

Recursion as control flow

FP commonly uses recursion where imperative languages use loops.
Key ingredients:

- Base case: simplest input
- Recursive case: reduce the problem toward the base case

Why recursion fits FP:

- Works naturally with immutable lists
- Connects to inductive data definitions
- Enables structural decomposition via pattern matching

Tail recursion and performance

Tail recursion

- The recursive call is the last action in the function
- Some languages optimize it (turn it into a loop internally)

Practical reality:

- Haskell compilers often optimize tail recursion well
- Python does not optimize tail calls
- JavaScript engines vary; recursion depth can still be a concern

In non-optimizing runtimes, folds and loops may be preferred for large inputs.

Haskell: recursion on lists

A classic pattern: sum of a list.

```
1 sumList :: [Int] -> Int
2 sumList [] = 0
3 sumList (x:xs) = x + sumList xs
```

[] is the empty list. (x:xs) matches a non-empty list with head x and tail xs.

TypeScript: recursion (small inputs)

TypeScript can use recursion, but large recursion can overflow the call stack.

```
1 function sumList(xs: number[]): number {
2   if (xs.length === 0) return 0;
3   const [x, ...rest] = xs;
4   return x + sumList(rest);
5 }
6
7 console.log(sumList([1, 2, 3]));
```

In TypeScript, reduce is usually preferred for large arrays.

Python: recursion (small inputs)

```
1 def sum_list(xs):
2     if xs == []:
3         return 0
4     x, *rest = xs
5     return x + sum_list(rest)
6
7 print(sum_list([1, 2, 3]))
```

Python recursion has a depth limit; iterative or fold-style code is often used in production.

Practice 3: recursion vs fold

Task A: write factorial recursively in Haskell.

Task B: write sum of squares using a fold (reduce) in Haskell.

```
1 -- factorial n = 1 * 2 * ... * n
2 -- sumSquares [a,b,c] = a*a + b*b + c*c
```

Practice 3 Answer: recursion vs fold

Recursive factorial:

```haskell
factorial :: Int -> Int
factorial 0 = 1
factorial n = n * factorial (n - 1)
```

Sum of squares using foldl:

```haskell
sumSquares :: [Int] -> Int
sumSquares xs = foldl (\acc x -> acc + x*x) 0 xs
```

foldl accumulates a running total; the lambda updates acc using x*x.

Lists and pattern matching

Pattern matching

Pattern matching means selecting behavior based on the shape of data.
Common uses:

- Decompose lists into empty vs non-empty
- Decompose variants (Option/Maybe, Result/Either)
- Make impossible states unrepresentable via types (in strongly typed FP)

Haskell uses pattern matching heavily; modern Python has match; TypeScript uses discriminated unions.

Haskell: list patterns

```haskell
describe :: [a] -> String
describe [] = "empty"
describe [x] = "one element"
describe (x:y:_) = "two or more"
```

Patterns are checked top to bottom. Underscore means the value is ignored.

Haskell: Maybe for safe results

Maybe a represents an optional value.

```haskell
safeHead :: [a] -> Maybe a
safeHead [] = Nothing
safeHead (x:_) = Just x
```

safeHead avoids runtime errors on empty lists by returning Nothing.

TypeScript: discriminated union for safe results

TypeScript can model Maybe with a tagged union.

```
type Option<T> =
  | { tag: "none" }
  | { tag: "some"; value: T };

function safeHead<T>(xs: T[]): Option<T> {
  if (xs.length === 0) return { tag: "none" };
  return { tag: "some", value: xs[0] };
}

const r = safeHead([10, 20]);
if (r.tag === "some") console.log(r.value);
```

The tag field lets the program narrow types safely.

Python: match (conceptual pattern matching)

Python match can destructure shapes and check cases.

```
1 def describe(xs):
2     match xs:
3         case []:
4             return "empty"
5         case [x]:
6             return "one element"
7         case [x, y, *rest]:
8             return "two or more"
```

Python pattern matching is runtime-based; Haskell is type-driven and exhaustive checking is stronger.

Practice 4: safe decomposition with pattern matching

Task A: implement safeTail in Haskell:

```haskell
-- safeTail [] = Nothing
-- safeTail (x:xs) = Just xs
```

Task B: implement safeTail in TypeScript using Option$< T >$.

- Use pattern matching in Haskell
- Use tag narrowing in TypeScript

Practice 4 Answer: safe decomposition with pattern matching

Haskell solution:

```haskell
safeTail :: [a] -> Maybe [a]
safeTail [] = Nothing
safeTail (_:xs) = Just xs
```

TypeScript solution:

```typescript
type Option<T> =
  | { tag: "none" }
  | { tag: "some"; value: T };

function safeTail<T>(xs: T[]): Option<T[]> {
  if (xs.length === 0) return { tag: "none" };
  return { tag: "some", value: xs.slice(1) };
}
```

Both versions avoid errors by making the empty case explicit.

Strict vs lazy evaluation

Strict evaluation (eager)

Strict evaluation means:

- Function arguments are evaluated before the function runs
- Intermediate collections are often fully computed

Typical strict languages:

- TypeScript/JavaScript
- Python (mostly strict, with generators as a lazy feature)
- Java

Strict evaluation is often easier to predict, but can do unnecessary work if results are only partially needed.

Lazy evaluation (conceptual)

Lazy evaluation means:

- Expressions are evaluated only when their values are needed
- Infinite data structures can be represented (infinite lists)
- Computation can stop early when enough results are produced

Haskell is the major mainstream lazy language.
Laziness can improve composability and performance, but can also make space usage harder to predict.

Haskell: infinite list and take

Haskell can define an infinite list and still produce a finite result.

```haskell
nums :: [Int]
nums = [1..]              -- infinite list

first5 :: [Int]
first5 = take 5 nums   -- [1,2,3,4,5]
```

take only evaluates as much of nums as needed to produce 5 elements.

Python: generators as lazy sequences

Generators are lazy: values are produced on demand.

```
1  def naturals():
2      n = 1
3      while True:
4          yield n
5          n += 1
6
7  g = naturals()
8  print(next(g))
9  print(next(g))
10 print(next(g))
```

next pulls one value; the generator does not produce an infinite list all at once.

TypeScript: iterators as lazy sequences

Iterators and generators can model laziness.

```
1  function* naturals(): Generator<number> {
2    let n = 1;
3    while (true) {
4      yield n;
5      n = n + 1;
6    }
7  }
8
9  const g = naturals();
10 console.log(g.next().value);
11 console.log(g.next().value);
12 console.log(g.next().value);
```

Practice 5: predict lazy behavior

Predict the output order in Python.

```
1  def gen():
2      print("start")
3      yield 1
4      print("after 1")
5      yield 2
6      print("after 2")
7
8  g = gen()
9  print("A")
10 print(next(g))
11 print("B")
12 print(next(g))
13 print("C")
```

- Which prints happen at generator creation time
- Which prints happen at each next call

Practice 5 Answer: predict lazy behavior

Output order:

```
A
start
1
B
after 1
2
C
```

Explanation

- Creating g = gen() does not execute the body, it creates a generator object.
- The body runs when next(g) is called.
- The generator resumes from the last yield on each next call.
- after 2 is not printed because the generator is not advanced past the second yield.

FP in the wild

Functional style in TypeScript and Python

You can use FP principles in non-FP languages by focusing on:

- Pure functions for core logic
- Immutable update patterns for state
- Small functions composed into pipelines
- Explicit error modeling (Option/Result style) instead of throwing everywhere

This style is common in front-end state management and data transformation code.

TypeScript: reducer pattern for state updates

Reducer: (state, action)$\rightarrow$ newState

```ts
type State = { count: number };
type Action =
  | { type: "inc" }
  | { type: "add"; n: number };
function reducer(s: State, a: Action): State {
  switch (a.type) {
    case "inc":
      return { ...s, count: s.count + 1 };
    case "add":
      return { ...s, count: s.count + a.n };
  }
}
let s: State = { count: 0 };
s = reducer(s, { type: "inc" });
s = reducer(s, { type: "add", n: 5 });
console.log(s.count);
```

No mutation of s in place; each step returns a new state object.

Python: pure transform pipeline

A small data pipeline written as pure functions.

```
1  def normalize(xs):
2      return [x.strip().lower() for x in xs]
3
4  def keep_nonempty(xs):
5      return [x for x in xs if x != ""]
6
7  def to_lengths(xs):
8      return [len(x) for x in xs]
9
10 data = ["  Ada  ", "", "Lambda", " FP "]
11 result = to_lengths(keep_nonempty(normalize(data)))
12 print(result)
```

Each function is easy to test and has no side effects.

Industry: data pipelines

FP patterns are common in data processing:

- Extract: read input sources
- Transform: map/filter/group/fold style logic
- Load: write outputs

Why FP fits:

- Transform stages are naturally pure
- Pipelines are composable and testable
- Parallel execution is easier when transforms are pure

Practice 6: reducer reasoning (TypeScript)

Predict the final count.

```ts
type State = { count: number };
type Action =
  | { type: "inc" }
  | { type: "add"; n: number }
  | { type: "reset" };
function reducer(s: State, a: Action): State {
  switch (a.type) {
    case "inc": return { ...s, count: s.count + 1 };
    case "add": return { ...s, count: s.count + a.n };
    case "reset": return { count: 0 };
  }
}
let s: State = { count: 1 };
s = reducer(s, { type: "inc" });
s = reducer(s, { type: "add", n: 10 });
s = reducer(s, { type: "reset" });
s = reducer(s, { type: "add", n: 3 });
console.log(s.count);
```

Practice 6 Answer: reducer reasoning (TypeScript)

Step-by-step:

- start count $= 1$
- inc $\rightarrow 2$
- add 10 $\rightarrow 12$
- reset $\rightarrow 0$
- add 3 $\rightarrow 3$

Final output:

- 3

Pure FP in Haskell

Haskell viewpoint: purity by default

Haskell enforces purity:

- Most functions cannot perform I/O
- Effects are modeled in types (IO a)
- Pure code stays pure; effectful code is explicit

This creates a strong separation:

- Pure transformations are deterministic and testable
- I/O is structured and localized

Haskell: types communicate intent

```haskell
-- pure
inc :: Int -> Int
inc x = x + 1

-- pure
sumSquares :: [Int] -> Int
sumSquares xs = foldl (\acc x -> acc + x*x) 0 xs

-- effectful
printInc :: Int -> IO ()
printInc x = print (inc x)
```

IO () indicates an effectful computation that returns no meaningful value.

Haskell: IO boundary example

One common structure: read input, pure transform, print output.

```haskell
parseInt :: String -> Int
parseInt s = read s

compute :: Int -> Int
compute n = n * n

main :: IO ()
main = do
  line <- getLine
  let n = parseInt line
  let r = compute n
  print r
```

parseInt and compute are pure; main handles I/O.

Why types help in pure FP

In a pure FP language, types provide strong guarantees:

- If a function has type Int $\rightarrow$ Int, it cannot print or mutate global state
- If a function returns IO a, it may do effects
- Algebraic data types help represent structured domain states

This reduces accidental coupling between business logic and I/O logic.

Practice 7: classify pure vs effectful (Haskell)

For each signature, decide whether it can perform I/O effects.

```haskell
a :: String -> Int
b :: String -> IO Int
c :: IO String
d :: [Int] -> Int
e :: Int -> IO ()
```

- Which are pure transformations
- Which are effectful computations

Practice 7 Answer: classify pure vs effectful (Haskell)

Pure transformations:

- a :: String $\rightarrow$ Int
- d :: [Int] $\rightarrow$ Int

Effectful computations:

- b :: String $\rightarrow$ IO Int
- c :: IO String
- e :: Int $\rightarrow$ IO ()

Rule: presence of IO in the type indicates effects are allowed.

FP for concurrency and parallelism

Why immutability helps concurrency

Concurrency bugs often come from shared mutable state:

- Two threads update the same data without coordination
- Interleavings cause inconsistent results
- Debugging is hard because schedules vary

Immutability helps:

- Shared reads are safe
- Data races are reduced
- Parallel map/reduce becomes easier to reason about

TypeScript: parallel map with Promise.all

If work is pure and independent, it composes well.

```
function work(x: number): Promise<number> {
  return new Promise(resolve => setTimeout(() => resolve(x *
  ↪  x), 10));
}

async function run(): Promise<void> {
  const xs = [1, 2, 3, 4];
  const ys = await Promise.all(xs.map(work));
  console.log(ys);
}

run();
```

work has no shared state, so running many instances is safe.

Python: parallel map style idea

In Python, similar patterns exist with executors.

```
1 import concurrent.futures as cf
2
3 def work(x):
4     return x * x
5
6 xs = [1, 2, 3, 4]
7 with cf.ThreadPoolExecutor() as ex:
8     ys = list(ex.map(work, xs))
9 print(ys)
```

Pure functions reduce the risk of race conditions and make results deterministic.

Practice 8: make concurrency safer

You want to parallelize a computation.

Starting point has shared mutation:

```
let total = 0;
function addWork(x: number): void {
  total = total + x * x;
}
```

Task:

- Explain why this is risky under concurrency
- Refactor into a pure function and a separate reduce step

Practice 8 Answer: make concurrency safer

Why it is risky:

- total is shared mutable state
- concurrent updates can interleave and lose increments

Refactor:

```
function work(x: number): number {
  return x * x;
}

const xs = [1, 2, 3, 4];
const squares = xs.map(work);
const total = squares.reduce((acc, v) => acc + v, 0);

console.log(total);
```

If work becomes async, use Promise.all to compute squares, then reduce.

Industry angle

FP for front-end state management

Front-end apps manage evolving state:

- user actions produce events
- events transform state
- UI is a pure function of state in many frameworks

Functional approach:

- represent updates as pure reducers
- avoid in-place mutation
- use time-travel debugging and replay of actions

This style appears in Redux-like patterns and in general React state update thinking.

FP for data processing and observability

Logs and telemetry often use functional pipelines:

- parse input lines
- filter relevant events
- map to structured records
- reduce to metrics (counts, percentiles, histograms)

FP helps because:

- transformations are simple and reusable
- functions are easy to test
- parallelization is natural when transforms are pure

Common FP tradeoffs in practice

Tradeoffs:

- Performance: copying data can cost time and memory if done naively
- Learning curve: recursion, higher-order functions, and types can be new
- Debugging: stacks and indirection can be confusing at first

Practical middle ground in industry:

- Use functional style for core logic and transformations
- Use mutation internally in performance-critical code, with clear boundaries
- Keep APIs pure where possible

Review

Summary: what to remember

- FP emphasizes expressions, pure functions, and immutable data
- Referential transparency supports safe reasoning and refactoring
- Higher-order functions enable reusable control patterns like map/filter/fold
- Recursion and pattern matching align with inductive data structures
- Haskell enforces purity and makes effects explicit with IO types
- Laziness can avoid unnecessary work and support infinite structures
- Functional style is widely used in TypeScript/Python for pipelines and state reducers
- Immutability simplifies reasoning about concurrency and parallelism

Review questions

- What is a pure function, and what counts as a side effect
- Why does immutability help testing and concurrency
- When should you prefer map/filter/reduce over a manual loop
- What is the base case in a recursive function and why is it required
- What does pattern matching give you that nested if statements do not
- In Haskell, what does IO in a type mean
- What is one advantage and one risk of laziness